Lecture 4

C# Methods

A **method** is a block of code which only runs when it is called.

You can pass data, known as parameters, into a method.

Methods are used to perform certain actions, and they are also known as **functions**.

Example

Inside Main(), call the myMethod() method:

```
static void MyMethod()
{
  Console.WriteLine("I just got executed!");
}

static void Main(string[] args)
{
  MyMethod();
}

// Outputs "I just got executed!"
```

C# Method Parameters

Parameters and Arguments

Information can be passed to methods as parameter. Parameters act as variables inside the method.

They are specified after the method name, inside the parentheses. You can add as many parameters as you want, just separate them with a comma.

The following example has a method that takes a string called **fname** as parameter. When the method is called, we pass along a first name, which is used inside the method to print the full name:

Example

```
static void MyMethod(string fname, int no)
{
  Console.WriteLine(fname + " index is: " + no);
}

static void Main(string[] args)
{
  MyMethod("abc" , 5);
  MyMethod("xyz" , 2);
  MyMethod("efg" , 8);
}

// abc index is: 5
// xyz index is: 2
// efg index is: 8
```

Default Parameter Value

You can also use a default parameter value, by using the equals sign (=). If we call the method without an argument, it uses the default value ("Norway"):

Example

```
static void MyMethod(string country = "Norway")
{
  Console.WriteLine(country);
}

static void Main(string[] args)
{
  MyMethod("Sweden");
  MyMethod("India");
  MyMethod();
  MyMethod("USA");
}

// Sweden
// India
// Norway
// USA
```

Return Values

The void keyword, used in the examples above, indicates that the method should not return a value. If you want the method to return a value, you can use a primitive data type (such as int or double) instead of void, and use the return keyword inside the method:

Example

```
static int MyMethod(int x)
{
```

```
  return 5 + x;
}

static void Main(string[] args)
{
  Console.WriteLine(MyMethod(3));
}

// Outputs 8 (5 + 3)
```

Named Arguments

It is also possible to send arguments with the *key: value* syntax.

That way, the order of the arguments does not matter:

Example

```
static void MyMethod(string child1, string child2, string child3)
{
  Console.WriteLine("The youngest child is: " + child3);
}

static void Main(string[] args)
{
  MyMethod(child3: "John", child1: "Liam", child2: "Liam");
}

// The youngest child is: John
```

Method Overloading

With **method overloading**, multiple methods can have the same name with different parameters:

Example

```
static int PlusMethod(int x, int y)
{
  return x + y;
}

static double PlusMethod(double x, double y)
{
  return x + y;
}

static void Main(string[] args)
{
  int myNum1 = PlusMethod(8, 5);
  double myNum2 = PlusMethod(4.3, 6.26);
  Console.WriteLine("Int: " + myNum1);
  Console.WriteLine("Double: " + myNum2);
```

C# Classes and Objects

Everything in C# is associated with classes and objects, along with its attributes and methods. For example: in real life, a car is an object. The car has **attributes**, such as weight and color, and **methods**, such as drive and brake.

Example

```
class Car
{
 string color = "red";


 static void Main(string[] args)
 {
  Car myObj = new Car();
  Console.WriteLine(myObj.color);
 }
}
```

Output = red

Using Multiple Classes

You can also create an object of a class and access it in another class. This is often used for better organization of classes (one class has all the fields and methods, while the other class holds the Main() method (code to be executed)).

- prog2.cs
- prog.cs

prog2.cs

```
class Car
{
 public string color = "red";
}
```

prog.cs

```
class Program
```

```
{
  static void Main(string[] args)
  {
    Car myObj = new Car();
    Console.WriteLine(myObj.color);
  }
}
```

Output= red

C# Constructors

Example

```
class Car
{
  public string model;

  // Create a class constructor with a parameter
  public Car(string modelName)
  {
    model = modelName;
  }

  static void Main(string[] args)
  {
    Car Ford = new Car("Mustang");
    Console.WriteLine(Ford.model);
```

```
  }
}


// Outputs "Mustang"
```

If there are no parameter then you will initialize value inside constructor.

C# Access Modifiers

Public:

The code is accessible for all classes

Private:

The code is only accessible within the same class

Protected:

The code is accessible within the same class, or in a class that is inherited from that class.

Internal:

The code is only accessible within its own assembly, but not from another assembly

C# Properties (Get and Set)

The meaning of **Encapsulation**, is to make sure that "sensitive" data is hidden from users. To achieve this, you must:

- declare fields/variables as private
- provide public get and set methods, through **properties**, to access and update the value of a private field

A property is like a combination of a variable and a method, and it has two methods: a get and a set method.

Example

```
class Person

{
```

```
  private string name; // field
  public string Name   // property
  {
    get { return name; }
    set { name = value; }
  }
}


class Program
{
  static void Main(string[] args)
  {
    Person myObj = new Person();
    myObj.Name = "Liam";
    Console.WriteLine(myObj.Name);
  }
}
```

OUTPUT?

Automatic Properties (Short Hand)

C# also provides a way to use short-hand / automatic properties, where you do not have to define the field for the property, and you only have to write get; and set; inside the property.

The following example will produce the same result as the example above. The only difference is that there is less code:

Example

Using automatic properties:

```
class Person
{
  public string Name  // property
  { get; set; }
}


class Program
{
  static void Main(string[] args)
  {
    Person myObj = new Person();
    myObj.Name = "Liam";
    Console.WriteLine(myObj.Name);
  }
}
```

OUTPUT= Liam

C# Inheritance

In C#, it is possible to inherit fields and methods from one class to another. We group the "inheritance concept" into two categories:

- **Derived Class** (child) - the class that inherits from another class
- **Base Class** (parent) - the class being inherited from

To inherit from a class, use the : symbol.

In the example below, the Car class (child) inherits the fields and methods from the Vehicle class (parent):

Example

```
class Vehicle  // base class (parent)
{
  public string brand = "Ford";  // Vehicle field
  public void honk()             // Vehicle method
  {
    Console.WriteLine("Tuut, tuut!");
  }
}

class Car : Vehicle  // derived class (child)
{
  public string modelName = "Mustang";  // Car field
}

class Program
{
  static void Main(string[] args)
  {
    // Create a myCar object
    Car myCar = new Car();

    // Call the honk() method (From the Vehicle class) on the myCar object
    myCar.honk();
```

```
  // Display the value of the brand field (from the Vehicle class) and the value of the modelName from the Car class
  Console.WriteLine(myCar.brand + " " + myCar.modelName);
 }
}
```

OUTPUT?

C# Polymorphism

Polymorphism means "many forms", and it occurs when we have many classes that are related to each other by inheritance. **Polymorphism** uses those methods to perform different tasks. This allows us to perform a single action in different ways.

Example

```
class Animal  // Base class (parent)
{
 public void animalSound()
 {
  Console.WriteLine("The animal makes a sound");
 }
}


class Pig : Animal  // Derived class (child)
{
 public void animalSound()
 {
  Console.WriteLine("The pig says: mein meinnnnnn
 }
}
```

```
class Dog : Animal  // Derived class (child)
{
  public void animalSound()
  {
    Console.WriteLine("The dog says: bow wow");
  }
}

class Program
{
  static void Main(string[] args)
  {
    Animal myAnimal = new Animal();  // Create a Animal object
    Animal myPig = new Pig();  // Create a Pig object
    Animal myDog = new Dog();  // Create a Dog object

    myAnimal.animalSound();
    myPig.animalSound();
    myDog.animalSound();
  }
}
```

The output will be:

```
The animal makes a sound
The animal makes a sound
The animal makes a sound
```

Not The Expected Output

The output from the example above was probably not what you expected. That is because the base class method overrides the derived class method, when they share the same name.

However, C# provides an option to override the base class method, by adding the virtual keyword to the method inside the base class, and by using the override keyword for each derived class methods.

Example

```
class Animal  // Base class (parent)
{
  public virtual void animalSound()
  {
    Console.WriteLine("The animal makes a sound");
  }
}


class Pig : Animal  // Derived class (child)
{
  public override void animalSound()
  {
    Console.WriteLine("The pig says: wee wee");
  }
```

```
}


class Dog : Animal  // Derived class (child)
{
  public override void animalSound()
  {
    Console.WriteLine("The dog says: bow wow");
  }
}


class Program
{
  static void Main(string[] args)
  {
    Animal myAnimal = new Animal();  // Create a Animal object
    Animal myPig = new Pig();  // Create a Pig object
    Animal myDog = new Dog();  // Create a Dog object


    myAnimal.animalSound();
    myPig.animalSound();
    myDog.animalSound();
  }
}
```

The output will be:

```
The animal makes a sound
The Goat says: baa baa
The dog says: bow wow
```

C# Multi-Level Inheritance

Generally, c# supports only **single inheritance** that means a class can only inherit from one base class. However, in c# the inheritance is transitive and it allows you to define a hierarchical inheritance for a set of types and it is called a multi-level inheritance.

For example, suppose if class **C** is derived from class **B**, and class **B** is derived from class **A**, then class **C** inherits the members declared in both class **B** and class **A**.

```
public class A
{
// Implementation
}
public class B : A
{
// Implementation
}
public class C : B
{
// Implementation
}
```

C# Multiple Inheritance

C# supports only **single inheritance** that means a class can only inherit from one base class. In case, if we try to inherit a class from multiple base classes, then we will get compile-time errors.

For example, if class **C** is trying to inherit from Class **A** and **B** at the same time, then we will get a compile-time error because multiple inheritance is not allowed in c#.

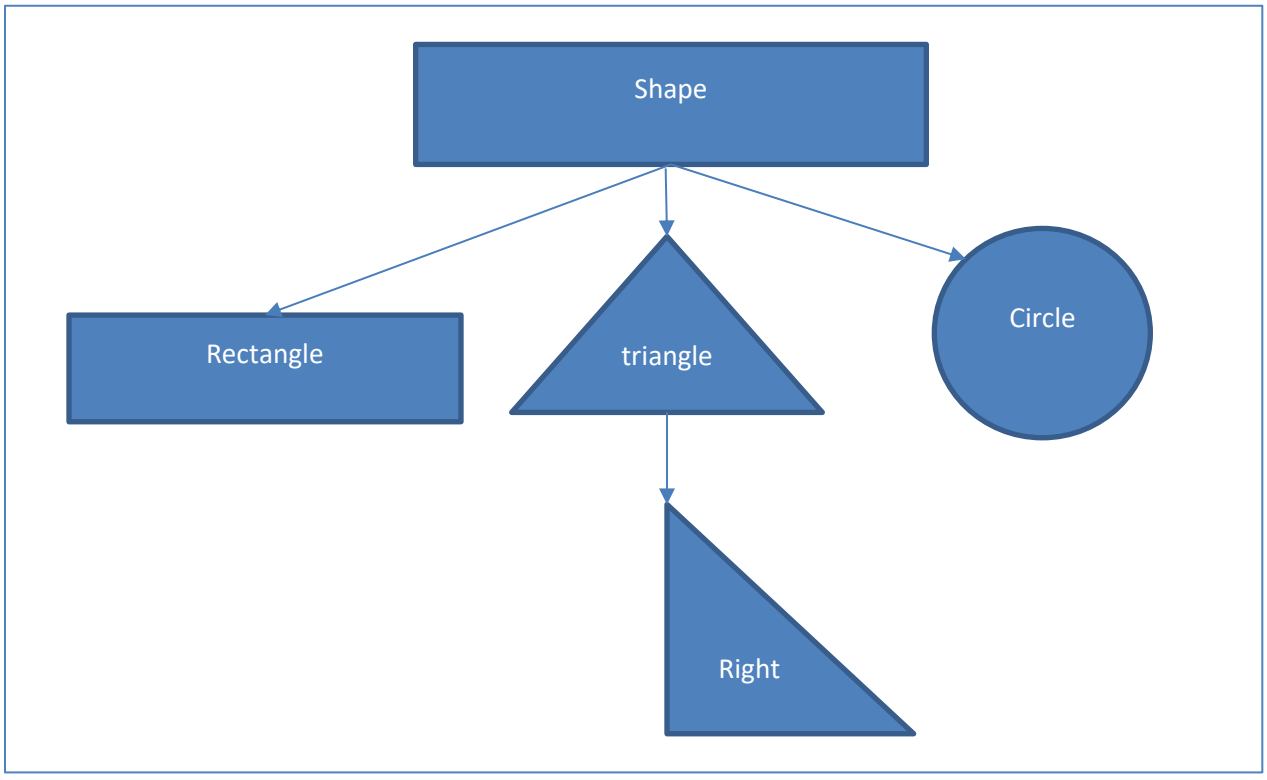
```
public class A
{
// Implementation
}
```

```
public class B
{
// Implementation
}
public class C : A, B
{
// Implementation
}
```

If you observe above code snippet, class **C** is trying to inherit properties from both classes **A** and **B** at the same time which will lead to compile-time error like "**Class C cannot have multiple classes: A and B**".

As discussed, **multi-level** inheritance is supported in c# but **multiple** inheritance is not supported. In case, if you want to implement **multiple** inheritance in c#, that we can achieve by using **interfaces**

Lab Task:



1. Based upon the figure given above identify and implement the classes, base class, derived classes and type of inheritance used.
2. You are required to calculate the area of each shape which you will do by use of function overriding.

3. All the input and output values must be through Property functions (set & get).
4. Print area of each shape, input values must be user defined and print sequence of constructor and destructor calling w.r.t to inheritance.